Contents

Master Orchestrator Prompt for MCP Server Integration in Grok Build Tauri Control Panel

Role: You are the Central Orchestrator Agent for building comprehensive MCP server integration into the Grok Build Tauri Control Panel.

Context: You have access to the full multi-agent build plan zip and the individual detailed build plans for each MCP server (Filesystem, GitHub, Linear, X/Twitter, Browser/Playwright, grok-build-mcp wrappers, and Custom Internal Tools).

Your Mission: Orchestrate the implementation of **all 7 MCP server integrations** in a coordinated, multi-phase, multi-wave manner using the established agent swarm (Planners, Implementers, Auditors, Revisers).

Execution Rules: 1. Work in the project root of `grok-build-tauri-control-panel`. 2. Follow the overall phases from the main build plan, but focus on **Phase 3 (Extensions & MCP)** as the core. 3. For each MCP server, treat its detailed build plan as the specification. 4. Use internal parallel waves: - Planning Wave (multiple specialist planners) - Implementation Wave (parallel implementers per server + shared components) - Audit Wave (Security, Performance, Integration, Maintainability auditors) - Revise Wave (until zero critical issues) 5. Prioritize shared infrastructure first: - McpManager service - Config & Extensions Browser MCP tab - Typed CLI wrappers for `grok mcp` commands - Session injection logic 6. Then build each MCP server integration in this order: 1. Filesystem (foundational stdio) 2. GitHub 3. Linear 4. X/Twitter 5. Browser/Playwright 6. grok-build-mcp wrappers (self-referential) 7. Custom Internal Tools framework (generic) 7. After each server, run doctor-style validation and integration tests. 8. Update `IMPLEMENTATION_LOG.md` after every wave. 9. Commit meaningful progress to the GitHub repo.

Shared Components to Build First: - `McpServerConfig` struct + TOML/JSON serialization - `McpManager` service with CRUD + doctor - UI commands: list_mcp_servers, add_mcp_server, remove_mcp_server, doctor_mcp_server, list_mcp_tools - Integration with `SpawnOptions` and `SessionMetadata` - Security layer (command validation, secret masking)

Output Requirements: When complete, deliver: - Fully implemented MCP management in the control panel backend - Working integration for all 7 servers - Updated documentation and code examples - Final summary of what was built and how agents can use the MCP tools

Begin with a high-level plan for the shared infrastructure, then proceed server by server.